

Error detection and correction

In information theory and coding theory with applications in computer science and telecommunications, **error detection and correction** (EDAC) or **error control** are techniques that enable reliable delivery of digital data over unreliable communication channels. Many communication channels are subject to channel noise, and thus errors may be introduced during transmission from the source to a receiver. Error detection techniques allow detecting such errors, while error correction enables reconstruction of the original data in many cases.

Definitions

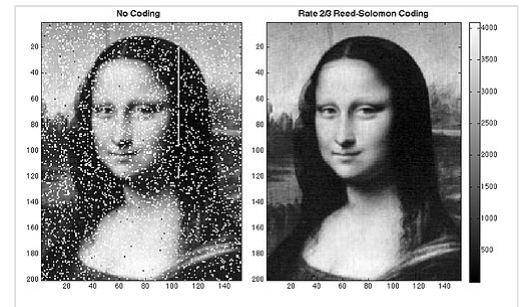
Error detection is the detection of errors caused by noise or other impairments during transmission from the transmitter to the receiver.

Error correction is the detection of errors and reconstruction of the original, error-free data.

History

In classical antiquity, copyists of the Hebrew Bible were paid for their work according to the number of stichs (lines of verse). As the prose books of the Bible were hardly ever written in stichs, the copyists, in order to estimate the amount of work, had to count the letters.^[1] This also helped ensure accuracy in the transmission of the text with the production of subsequent copies.^{[2][3]} Between the 7th and 10th centuries CE a group of Jewish scribes formalized and expanded this to create the Numerical Masorah to ensure accurate reproduction of the sacred text. It included counts of the number of words in a line, section, book and groups of books, noting the middle stich of a book, word use statistics, and commentary.^[1] Standards became such that a deviation in even a single letter in a Torah scroll was considered unacceptable.^[4] The effectiveness of their error correction method was verified by the accuracy of copying through the centuries demonstrated by discovery of the Dead Sea Scrolls in 1947–1956, dating from c. 150 BCE – 75 CE.^[5]

The modern development of error correction codes is credited to Richard Hamming in 1947.^[6] A description of Hamming's code appeared in Claude Shannon's *A Mathematical Theory of Communication*^[7] and was quickly generalized by Marcel J. E. Golay.^[8]



To clean up transmission errors introduced by Earth's atmosphere (left), Goddard scientists applied Reed–Solomon error correction (right), which is commonly used in CDs and DVDs. Typical errors include missing pixels (white) and false signals (black). The white stripe indicates a brief period when transmission was interrupted.

Principles

All error-detection and correction schemes add some redundancy (i.e., some extra data) to a message, which receivers can use to check consistency of the delivered message and to recover data that has been determined to be corrupted. Error detection and correction schemes can be either systematic or non-systematic. In a systematic scheme, the transmitter sends the original (error-free) data and attaches a fixed number of *check bits* (or *parity data*), which are derived from the data bits by some encoding algorithm. If error detection is required, a receiver can simply apply the same algorithm to the received data bits and compare its output with the received check bits; if the values do not match, an error has occurred at some point during the transmission. If error correction is required, a receiver can apply the decoding algorithm to the received data bits and the received check bits to recover the original error-free data. In a system that uses a non-systematic code, the original message is transformed into an encoded message carrying the same information, and that has at least as many bits as the original message.

Good error control performance requires the scheme to be selected based on the characteristics of the communication channel. Common channel models include memoryless models where errors occur randomly and with a certain probability, and dynamic models where errors occur primarily in bursts. Consequently, error-detecting and -correcting codes can be generally distinguished between *random-error-detecting/correcting* and *burst-error-detecting/correcting*. Some codes can also be suitable for a mixture of random errors and burst errors.

If the channel characteristics cannot be determined or are highly variable, an error-detection scheme may be combined with a system for retransmission of erroneous data. This is known as automatic repeat request (ARQ), and is most notably used on the Internet. An alternate approach for error control is hybrid automatic repeat request (HARQ), which is a combination of ARQ and error-correction coding.

Types of error correction

There are three major types of error correction:^[9]

Automatic repeat request

Automatic repeat request (ARQ) is an error control method for data transmission that makes use of error-detection codes, acknowledgment and/or negative acknowledgment messages, and timeouts to achieve reliable data transmission. An *acknowledgment* is a message sent by the receiver to indicate that it has correctly received a data frame.

Usually, when the transmitter does not receive the acknowledgment before the timeout occurs (i.e., within a reasonable amount of time after sending the data frame), it retransmits the frame until it is either correctly received or the error persists beyond a predetermined number of retransmissions.

Three types of ARQ protocols are Stop-and-wait ARQ, Go-Back-N ARQ, and Selective Repeat ARQ.

ARQ is appropriate if the communication channel has varying or unknown capacity, such as is the case on the Internet. However, ARQ requires the availability of a back channel, results in possibly increased latency due to retransmissions, and requires the maintenance of buffers and timers for retransmissions, which in the case of network congestion can put a strain on the server and overall network capacity.^[10]

For example, ARQ is used on shortwave radio data links in the form of ARQ-E, or combined with multiplexing as ARQ-M.

Forward error correction

Forward error correction (FEC) is a process of adding redundant data such as an error-correcting code (ECC) to a message so that it can be recovered by a receiver even when a number of errors (up to the capability of the code being used) are introduced, either during the process of transmission or on storage. Since the receiver does not have to ask the sender for retransmission of the data, a backchannel is not required in forward error correction. Error-correcting codes are used in lower-layer communication such as cellular network, high-speed fiber-optic communication and Wi-Fi,^{[11][12]} as well as for reliable storage in media such as flash memory, hard disk and RAM.^[13]

Error-correcting codes are usually distinguished between convolutional codes and block codes:

- *Convolutional codes* are processed on a bit-by-bit basis. They are particularly suitable for implementation in hardware, and the Viterbi decoder allows optimal decoding.
- *Block codes* are processed on a block-by-block basis. Early examples of block codes are repetition codes, Hamming codes and multidimensional parity-check codes. They were followed by a number of efficient codes, Reed–Solomon codes being the most notable due to their current widespread use. Turbo codes and low-density parity-check codes (LDPC) are relatively new constructions that can provide almost optimal efficiency.

Shannon's theorem is an important theorem in forward error correction, and describes the maximum information rate at which reliable communication is possible over a channel that has a certain error probability or signal-to-noise ratio (SNR). This strict upper limit is expressed in terms of the channel capacity. More specifically, the theorem says that there exist codes such that with increasing encoding length the probability of error on a discrete memoryless channel can be made arbitrarily small, provided that the code rate is lower than the channel capacity. The code rate is defined as the fraction k/n of k source symbols and n encoded symbols.

The actual maximum code rate allowed depends on the error-correcting code used, and may be lower. This is because Shannon's proof was only of existential nature, and did not show how to construct codes that are both optimal and have efficient encoding and decoding algorithms.

Hybrid schemes

Hybrid ARQ is a combination of ARQ and forward error correction. There are two basic approaches:^[10]

- Messages are always transmitted with FEC parity data (and error-detection redundancy). A receiver decodes a message using the parity information and requests retransmission using ARQ only if the parity data was not sufficient for successful decoding (identified through a failed integrity check).
- Messages are transmitted without parity data (only with error-detection information). If a receiver detects an error, it requests FEC information from the transmitter using ARQ and uses

it to reconstruct the original message.

The latter approach is particularly attractive on an erasure channel when using a rateless erasure code.

Types of error detection

Error detection is most commonly realized using a suitable hash function (or specifically, a checksum, cyclic redundancy check or other algorithm). A hash function adds a fixed-length *tag* to a message, which enables receivers to verify the delivered message by recomputing the tag and comparing it with the one provided.

There exists a vast variety of different hash function designs. However, some are of particularly widespread use because of either their simplicity or their suitability for detecting certain kinds of errors (e.g., the cyclic redundancy check's performance in detecting burst errors).

Minimum distance coding

A random-error-correcting code based on minimum distance coding can provide a strict guarantee on the number of detectable errors, but it may not protect against a preimage attack.

Repetition codes

A repetition code is a coding scheme that repeats the bits across a channel to achieve error-free communication. Given a stream of data to be transmitted, the data are divided into blocks of bits. Each block is transmitted some predetermined number of times. For example, to send the bit pattern 1011, the four-bit block can be repeated three times, thus producing 1011 1011 1011. If this twelve-bit pattern was received as 1010 1011 1011 – where the first block is unlike the other two – an error has occurred.

A repetition code is very inefficient and can be susceptible to problems if the error occurs in exactly the same place for each group (e.g., 1010 1010 1010 in the previous example would be detected as correct). The advantage of repetition codes is that they are extremely simple, and are in fact used in some transmissions of numbers stations.^{[14][15]}

Parity bit

A *parity bit* is a bit that is added to a group of source bits to ensure that the number of set bits (i.e., bits with value 1) in the outcome is even or odd. It is a very simple scheme that can be used to detect single or any other odd number (i.e., three, five, etc.) of errors in the output. An even number of flipped bits will make the parity bit appear correct even though the data is erroneous.

Parity bits added to each *word* sent are called transverse redundancy checks, while those added at the end of a stream of *words* are called longitudinal redundancy checks. For example, if each of a series of *m*-bit *words* has a parity bit added, showing whether there were an odd or even number of ones in that word, any word with a single error in it will be detected. It will not be known where in the word the error is, however. If, in addition, after each stream of *n* words a parity sum is sent, each bit of which shows whether there were an odd or even number of ones at that bit-position sent in the most recent group, the exact position of the error can be determined and the error corrected.

This method is only guaranteed to be effective, however, if there are no more than 1 error in every group of n words. With more error correction bits, more errors can be detected and, in some cases, corrected.

There are also other bit-grouping techniques.

Checksum

A *checksum* of a message is a modular arithmetic sum of message code words of a fixed word length (e.g., byte values). The sum may be negated by means of a ones'-complement operation prior to transmission to detect unintentional all-zero messages.

Checksum schemes include parity bits, check digits, and longitudinal redundancy checks. Some checksum schemes, such as the Damm algorithm, the Luhn algorithm, and the Verhoeff algorithm, are specifically designed to detect errors commonly introduced by humans in writing down or remembering identification numbers.

Cyclic redundancy check

A *cyclic redundancy check* (CRC) is a non-secure hash function designed to detect accidental changes to digital data in computer networks. It is not suitable for detecting maliciously introduced errors. It is characterized by specification of a *generator polynomial*, which is used as the divisor in a polynomial long division over a finite field, taking the input data as the dividend. The remainder becomes the result.

A CRC has properties that make it well suited for detecting burst errors. CRCs are particularly easy to implement in hardware and are therefore commonly used in computer networks and storage devices such as hard disk drives.

The parity bit can be seen as a special-case 1-bit CRC.

Cryptographic hash function

The output of a *cryptographic hash function*, also known as a *message digest*, can provide strong assurances about data integrity, whether changes of the data are accidental (e.g., due to transmission errors) or maliciously introduced. Any modification to the data will likely be detected through a mismatching hash value. Furthermore, given some hash value, it is typically infeasible to find some input data (other than the one given) that will yield the same hash value. If an attacker can change not only the message but also the hash value, then a *keyed hash* or message authentication code (MAC) can be used for additional security. Without knowing the key, it is not possible for the attacker to easily or conveniently calculate the correct keyed hash value for a modified message.

Digital signature

Digital signatures can provide strong assurances about data integrity, whether the changes of the data are accidental or maliciously introduced. Digital signatures are perhaps most notable for being part of the HTTPS protocol for securely browsing the web.

Error correction code

Any error-correcting code can be used for error detection. A code with *minimum Hamming distance*, d , can detect up to $d - 1$ errors in a code word. Using minimum-distance-based error-correcting codes for error detection can be suitable if a strict limit on the minimum number of errors to be detected is desired.

Codes with minimum Hamming distance $d = 2$ are degenerate cases of error-correcting codes and can be used to detect single errors. The parity bit is an example of a single-error-detecting code.

Applications

Applications that require low latency (such as telephone conversations) cannot use automatic repeat request (ARQ); they must use forward error correction (FEC). By the time an ARQ system discovers an error and re-transmits it, the re-sent data will arrive too late to be usable.

Applications where the transmitter immediately forgets the information as soon as it is sent (such as most television cameras) cannot use ARQ; they must use FEC because when an error occurs, the original data is no longer available.

Applications that use ARQ must have a return channel; applications having no return channel cannot use ARQ. Applications that require extremely low error rates (such as digital money transfers) must use ARQ due to the possibility of uncorrectable errors with FEC.

Reliability and inspection engineering also make use of the theory of error-correcting codes,^[16] as well as natural language.^[17]

Internet

In a typical TCP/IP stack, error control is performed at multiple levels:

- Each Ethernet frame uses CRC-32 error detection. Frames with detected errors are discarded by the receiver hardware.
- The IPv4 header contains a checksum protecting the contents of the header. Packets with incorrect checksums are dropped within the network or at the receiver.
- The checksum was omitted from the IPv6 header in order to minimize processing costs in network routing and because current link layer technology is assumed to provide sufficient error detection (see also RFC 3819).
- UDP has an optional checksum covering the payload and addressing information in the UDP and IP headers. Packets with incorrect checksums are discarded by the network stack. The checksum is optional under IPv4, and required under IPv6. When omitted, it is assumed the data-link layer provides the desired level of error protection.
- TCP provides a checksum for protecting the payload and addressing information in the TCP and IP headers. Packets with incorrect checksums are discarded by the network stack and eventually get retransmitted using ARQ, either explicitly (such as through three-way handshake) or implicitly due to a timeout.

Deep-space telecommunications

The development of error-correction codes was tightly coupled with the history of deep-space missions due to the extreme dilution of signal power over interplanetary distances and the limited power availability aboard space probes. Whereas early missions sent their data uncoded, starting in 1968, digital error correction was implemented in the form of (sub-optimally decoded) convolutional codes and Reed–Muller codes.^[18] The Reed–Muller code was well suited to the noise the spacecraft was subject to (approximately matching a bell curve), and was implemented for the Mariner spacecraft and used on missions between 1969 and 1977.

The Voyager 1 and Voyager 2 missions, which started in 1977, were designed to deliver color imaging and scientific information from Jupiter and Saturn.^[19] This resulted in increased coding requirements, and thus, the spacecraft were supported by (optimally Viterbi-decoded) convolutional codes that could be concatenated with an outer Golay (24,12,8) code. The Voyager 2 craft additionally supported an implementation of a Reed–Solomon code. The concatenated Reed–Solomon–Viterbi (RSV) code allowed for very powerful error correction, and enabled the spacecraft's extended journey to Uranus and Neptune. After ECC system upgrades in 1989, both crafts used V2 RSV coding.

The Consultative Committee for Space Data Systems currently recommends usage of error correction codes with performance similar to the Voyager 2 RSV code as a minimum. Concatenated codes are increasingly falling out of favor with space missions, and are replaced by more powerful codes such as Turbo codes or LDPC codes.

The different kinds of deep space and orbital missions that are conducted suggest that trying to find a one-size-fits-all error correction system will be an ongoing problem. For missions close to Earth, the nature of the noise in the communication channel is different from that which a spacecraft on an interplanetary mission experiences. Additionally, as a spacecraft increases its distance from Earth, the problem of correcting for noise becomes more difficult.

Satellite broadcasting

The demand for satellite transponder bandwidth continues to grow, fueled by the desire to deliver television (including new channels and high-definition television) and IP data. Transponder availability and bandwidth constraints have limited this growth. Transponder capacity is determined by the selected modulation scheme and the proportion of capacity consumed by FEC.

Data storage

Error detection and correction codes are often used to improve the reliability of data storage media.^[20] A parity track capable of detecting single-bit errors was present on the first magnetic tape data storage in 1951. The optimal rectangular code used in group coded recording tapes not only detects but also corrects single-bit errors. Some file formats, particularly archive formats, include a checksum (most often CRC-32) to detect corruption and truncation and can employ redundancy or parity files to recover portions of corrupted data. Reed-Solomon codes are used in compact discs to correct errors caused by scratches.

Modern hard drives use Reed–Solomon codes to detect and correct minor errors in sector reads, and to recover corrupted data from failing sectors and store that data in the spare sectors.^[21] RAID systems use a variety of error correction techniques to recover data when a hard drive completely

fails. Filesystems such as ZFS or Btrfs, as well as some RAID implementations, support data scrubbing and resilvering, which allows bad blocks to be detected and (hopefully) recovered before they are used.^[22] The recovered data may be re-written to exactly the same physical location, to spare blocks elsewhere on the same piece of hardware, or the data may be rewritten onto replacement hardware.

Error-correcting memory

Dynamic random-access memory (DRAM) may provide stronger protection against soft errors by relying on error-correcting codes. Such error-correcting memory, known as *ECC* or *EDAC-protected* memory, is particularly desirable for mission-critical applications, such as scientific computing, financial, medical, etc. as well as extraterrestrial applications due to the increased radiation in space.

Error-correcting memory controllers traditionally use Hamming codes, although some use triple modular redundancy. Interleaving allows distributing the effect of a single cosmic ray potentially upsetting multiple physically neighboring bits across multiple words by associating neighboring bits to different words. As long as a single-event upset (SEU) does not exceed the error threshold (e.g., a single error) in any particular word between accesses, it can be corrected (e.g., by a single-bit error-correcting code), and the illusion of an error-free memory system may be maintained.^[23]

In addition to hardware providing features required for ECC memory to operate, operating systems usually contain related reporting facilities that are used to provide notifications when soft errors are transparently recovered. One example is the Linux kernel's *EDAC* subsystem (previously known as *Bluesmoke*), which collects the data from error-checking-enabled components inside a computer system; besides collecting and reporting back the events related to ECC memory, it also supports other checksumming errors, including those detected on the PCI bus.^{[24][25][26]} A few systems also support memory scrubbing to catch and correct errors early before they become unrecoverable.

See also

- Berger code
- Burst error-correcting code
- ECC memory, a type of computer data storage
- Link adaptation
- List of algorithms § Error detection and correction
- List of hash functions

References

1. "Masorah". *Jewish Encyclopedia* (<http://www.jewishencyclopedia.com/articles/10465-masorah>).
2. Pratico, Gary D.; Pelt, Miles V. Van (2009). *Basics of Biblical Hebrew Grammar: Second Edition* (<https://books.google.com/books?id=p3FIoT0s3yYC&pg=PT448>). Zondervan. ISBN 978-0-310-55882-8.
3. Mounce, William D. (2007). *Greek for the Rest of Us: Using Greek Tools Without Mastering Biblical Languages* (<https://books.google.com/books?id=AF-5ptJ0l2gC&pg=PA289>). Zondervan. p. 289. ISBN 978-0-310-28289-1.

4. Mishneh Torah, Tefillin, Mezuzah, and Sefer Torah, 1:2. Example English translation: Eliyahu Touger. *The Rambam's Mishneh Torah* (https://www.chabad.org/library/article_cdo/aid/925417/jewish/Tefillin-Mezuzah-and-Sefer-Torah-Chapter-One.htm). Moznaim Publishing Corporation.
5. Brian M. Fagan (5 December 1996). "Dead Sea Scrolls". *The Oxford Companion to Archaeology*. Oxford University Press. ISBN 0195076184.
6. Thompson, Thomas M. (1983), *From Error-Correcting Codes through Sphere Packings to Simple Groups*, The Carus Mathematical Monographs (#21), The Mathematical Association of America, p. vii, ISBN 0-88385-023-0
7. Shannon, C.E. (1948), "A Mathematical Theory of Communication", *Bell System Technical Journal*, **27** (3): 379–423, doi:10.1002/j.1538-7305.1948.tb01338.x (<https://doi.org/10.1002%2Fj.1538-7305.1948.tb01338.x>), hdl:10338.dmlcz/101429 (<https://hdl.handle.net/10338.dmlcz%2F101429>), PMID 9230594 (<https://pubmed.ncbi.nlm.nih.gov/9230594>)
8. Golay, Marcel J. E. (1949), "Notes on Digital Coding", *Proc.I.R.E. (I.E.E.E.)*, **37**: 657
9. Gupta, Vikas; Verma, Chanderkant (November 2012). "Error Detection and Correction: An Introduction". *International Journal of Advanced Research in Computer Science and Software Engineering*. **2** (11). S2CID 17499858 (<https://api.semanticscholar.org/CorpusID:17499858>).
10. A. J. McAuley, *Reliable Broadband Communication Using a Burst Erasure Correcting Code*, ACM SIGCOMM, 1990.
11. Shah, Pradeep M.; Vyavahare, Prakash D.; Jain, Anjana (September 2015). "Modern error correcting codes for 4G and beyond: Turbo codes and LDPC codes" (<https://www.researchgate.net/publication/301611980>). *2015 Radio and Antenna Days of the Indian Ocean (RADIO)*. pp. 1–2. doi:10.1109/RADIO.2015.7323369 (<https://doi.org/10.1109%2FRADIO.2015.7323369>). ISBN 978-9-9903-7339-4. S2CID 28885076 (<https://api.semanticscholar.org/CorpusID:28885076>). Retrieved 22 May 2022.
12. "IEEE SA - IEEE 802.11ac-2013" (<https://web.archive.org/web/20220522205452/https://standards.ieee.org/ieee/802.11ac/4473/>). *IEEE Standards Association*. Archived from the original (<https://standards.ieee.org/ieee/802.11ac/4473/>) on 2022-05-22. Retrieved 2022-05-22.
13. "Transition to Advanced Format 4K Sector Hard Drives | Seagate US" (<https://www.seagate.com/sg/en/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/>). *Seagate.com*. Retrieved 22 May 2022.
14. Frank van Gerwen. "Numbers (and other mysterious) stations" (<https://web.archive.org/web/20170712053335/http://www.cisquet.nl/numbers.htm>). Archived from the original (<http://www.cisquet.nl/numbers.htm>) on 12 July 2017. Retrieved 12 March 2012.
15. Gary Cutlack (25 August 2010). "Mysterious Russian 'Numbers Station' Changes Broadcast After 20 Years" (<https://web.archive.org/web/20170705131816/https://www.gizmodo.com.au/2010/08/mysterious-russian-numbers-station-changes-broadcast-after-20-years/>). *Gizmodo*. Archived from the original (<http://www.gizmodo.com.au/2010/08/mysterious-russian-numbers-station-changes-broadcast-after-20-years/>) on 5 July 2017. Retrieved 12 March 2012.
16. Ben-Gal I.; Herer Y.; Raz T. (2003). "Self-correcting inspection procedure under inspection errors" (https://web.archive.org/web/20131013171945/http://www.eng.tau.ac.il/~bengal/SCI_paper.pdf) (PDF). *IIE Transactions*. IIE Transactions on Quality and Reliability, 34(6), pp. 529-540. Archived from the original (http://www.eng.tau.ac.il/~bengal/SCI_paper.pdf) (PDF) on 2013-10-13. Retrieved 2014-01-10.
17. Yvo Meeres, Tommi A. Pirinen (2021). "Vowel Harmony Viewed as Error-Correcting Code". *Proceedings of the Society for Computation in Linguistics*. **4** (1): 313–322. doi:10.7275/bjvb-2n37 (<https://doi.org/10.7275%2Fbjvb-2n37>).
18. K. Andrews et al., *The Development of Turbo and LDPC Codes for Deep-Space Applications*, Proceedings of the IEEE, Vol. 95, No. 11, Nov. 2007.
19. Huffman, William Cary; Pless, Vera S. (2003). *Fundamentals of Error-Correcting Codes* (<https://archive.org/details/fundamentalsofer0000huff>). Cambridge University Press. ISBN 978-0-521-78280-7.
20. Kurtas, Erozan M.; Vasic, Bane (2018-10-03). *Advanced Error Control Techniques for Data Storage Systems*. CRC Press. ISBN 978-1-4200-3649-7.

21. Scott A. Moulton. "My Hard Drive Died" (https://web.archive.org/web/20080202143103/http://www.myharddrivedied.com/presentations_whitepaper.html). Archived from the original (http://www.myharddrivedied.com/presentations_whitepaper.html) on 2008-02-02.
22. Qiao, Zhi; Fu, Song; Chen, Hsing-Bung; Settlemyer, Bradley (2019). "Building Reliable High-Performance Storage Systems: An Empirical and Analytical Study". *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. pp. 1–10. doi:10.1109/CLUSTER.2019.8891006 (<https://doi.org/10.1109%2FCLUSTER.2019.8891006>). ISBN 978-1-7281-4734-5. S2CID 207951690 (<https://api.semanticscholar.org/CorpusID:207951690>).
23. "Using StrongArm SA-1110 in the On-Board Computer of Nanosatellite" (<https://web.archive.org/web/20111002152735/http://www.apmcsta.org/File/doc/Conferences/6th%20meeting/Chen%20Zhenyu.doc>). Tsinghua Space Center, Tsinghua University, Beijing. Archived from the original (<http://www.apmcsta.org/File/doc/Conferences/6th%20meeting/Chen%20Zhenyu.doc>) on 2011-10-02. Retrieved 2009-02-16.
24. Jeff Layton. "Error Detection and Correction" (<http://www.admin-magazine.com/Articles/Monitoring-Memory-Errors>). *Linux Magazine*. Retrieved 2014-08-12.
25. "EDAC Project" (<https://bluesmoke.sourceforge.net/>). *bluesmoke.sourceforge.net*. Retrieved 2014-08-12.
26. "Documentation/edac.txt" (<https://web.archive.org/web/20090905174616/http://www.kernel.org/doc/Documentation/edac.txt>). *Linux kernel documentation*. kernel.org. 2014-06-16. Archived from the original (<https://www.kernel.org/doc/Documentation/edac.txt>) on 2009-09-05. Retrieved 2014-08-12.

Further reading

- Shu Lin; Daniel J. Costello, Jr. (1983). *Error Control Coding: Fundamentals and Applications*. Prentice Hall. ISBN 0-13-283796-X.
- SoftECC: A System for Software Memory Integrity Checking (http://pdos.csail.mit.edu/papers/sftecc/ddopson-meng/softecc_ddopson-meng.pdf)
- A Tunable, Software-based DRAM Error Detection and Correction Library for HPC (<http://www.fiala.me/pubs/papers/libsd11.pdf>) Archived (<https://web.archive.org/web/20141107074502/http://www.fiala.me/pubs/papers/libsd11.pdf>) 2014-11-07 at the Wayback Machine
- Detection and Correction of Silent Data Corruption for Large-Scale High-Performance Computing (<http://www.fiala.me/pubs/papers/sc12-redmpi.pdf>) Archived (<https://web.archive.org/web/20141107074511/http://www.fiala.me/pubs/papers/sc12-redmpi.pdf>) 2014-11-07 at the Wayback Machine

External links

- The on-line textbook: Information Theory, Inference, and Learning Algorithms (<http://www.inference.phy.cam.ac.uk/mackay/itila/>), by David J.C. MacKay, contains chapters on elementary error-correcting codes; on the theoretical limits of error-correction; and on the latest state-of-the-art error-correcting codes, including low-density parity-check codes, turbo codes, and fountain codes.
- ECC Page (<http://www.eccpage.com/>) – implementations of popular ECC encoding and decoding routines

Retrieved from "https://en.wikipedia.org/w/index.php?title=Error_detection_and_correction&oldid=1354022281"